Instituto Universitario de la Paz

Documentacion del proyecto de software: Sistema de Gestion de Tareas(To-Do List)

Integrantes:
Sebastian vasquez hoyos
Emmanuel Gutierrez Ortiz
Jesus Martinez
Esteban Antolinez

Materia:
Ingenieria de Software

Profesor:
Miguel Mateo Mont Lopez

Fecha de entrega:
08/04/2026

INDICE DEL CONTENIDO

1. **Introducción…….**

2. **Objetivos del Proyecto…….**

3. **Requerimientos del sistema (CRUD)……**

4. **Metodología de Desarrollo y control de versiones (Git Flow) …….**
   - **4.1.Estrategia de Ramas.**
   - **4.2.Registro de Commits y Merges.**

5. **Arquitectura de Diseño Técnico……**
   - **5.1.Stack Tecnológico.**
   - **5.2.Estructutra de Datos.**
   - **5.3.Diagrama de flujo.**

6. **Implementación y Pruebas…….**

7. **Conclusiones…….**

INTRODUCCION……

1. Descripción del Proyecto

El objetivo primordial de este proyecto es el desarrollo de un **Sistema de Gestión de Tareas (Task Management System)** diseñado para optimizar el flujo de trabajo personal o colaborativo. La solución permite el control total sobre el ciclo de vida de las tareas mediante una interfaz intuitiva y funcional.

Alcance Funcional (Requerimientos)

El sistema integra las operaciones fundamentales de persistencia de datos (CRUD), permitiendo al usuario:

- **Registro de Actividades:** Creación y almacenamiento de nuevas tareas.
- **Visualización Dinámica:** Listado detallada de las tareas pendientes y registradas.
- **Gestión de Estados:** Capacidad de conmutar tareas a estado "Completado" para seguimiento de progreso.
- **Depuración de Datos:** Eliminación definitiva de registros obsoletos o erróneos.

2. Marco Estratégico y Técnico

Para garantizar la escalabilidad y calidad del software, se han definido los siguientes pilares durante la fase de análisis:

Arquitectura de Trabajo (Git Flow)

Se ha establecido un **modelo de ramificación (Branching Strategy)** organizado para mantener la integridad del código fuente, diferenciando en dos ramas Pricipal y Indice.

Definición de Requisitos y Tecnologías

Tras un análisis de viabilidad, se seleccionó un stack tecnológico orientado a la eficiencia y la mantenibilidad:

1. **Requisitos Funcionales:** Definición de casos de uso y flujos de usuario.
2. **Stack Tecnológico:** Selección de lenguajes(html,css,jv, frameworks y herramientas de control de versiones idóneos para el cumplimiento de los objetivos técnicos.

2.Objetivos del Proyecto……

Objetivos del Proyecto

Objetivo General

Desarrollar una aplicación funcional para la gestión de tareas (To-Do List) que integre las operaciones fundamentales de manipulación de datos (CRUD), priorizando la aplicación de buenas prácticas en el proceso de desarrollo y el trabajo colaborativo.

Objetivos Específicos

- Implementar la lógica CRUD: Diseñar y codificar las funciones necesarias para crear, listar, actualizar el estado (marcar como completada) y eliminar tareas de manera eficiente.
- Aplicar una Estrategia de Ramificación (Git Flow): Gestionar el ciclo de vida del código mediante el uso de ramas independientes para cada funcionalidad, facilitando la integración continua y el orden del repositorio.
- Modularizar el Desarrollo: Estructurar el sistema de modo que cada funcionalidad (creación, listado, eliminación) opere de forma independiente, promoviendo la escalabilidad y el mantenimiento del código.
- Documentar el Proceso Técnico: Registrar las decisiones tomadas en cuanto a requisitos y tecnologías implementadas, demostrando la capacidad de análisis previo a la ejecución del software.
- Garantizar la Usabilidad: Proveer una interfaz (ya sea de consola o web) que permita al usuario interactuar con el sistema de gestión de forma clara e intuitiva.

3.Requerimientos del sistema (CRUD)……

sistema permite la persistencia de datos local (Client-side) y está preparado para una arquitectura de microservicios mediante el servidor Express adjunto.

1. Requerimientos Funcionales (Operaciones)

Operación	Descripción	Campos Involucrados
Crear (Create)	Permite al usuario registrar una nueva tarea mediante un formulario.	Título, Descripción (opcional), Prioridad, Categoría.
Leer (Read)	Visualización de la lista de tareas activas y filtrado por categorías o prioridad.	Todos los campos + ID + Fecha de creación.
Actualizar (Update)	Cambio de estado de la tarea (Pendiente / Completada) mediante un toggle.	Estado (completed).
Eliminar (Delete)	Remoción física de una tarea del listado tras confirmación del usuario.	ID de la tarea.

2. Estructura de Datos (Modelo)

Cada objeto de tarea dentro del sistema sigue el siguiente esquema técnico:

```
{
  "id": "Timestamp (Date.now())",
  "title": "String (Requerido)",
  "description": "String (Opcional)",
  "priority": "Enum ['alta', 'media', 'baja']",
  "category": "Enum ['personal', 'trabajo', 'estudios', 'otros']",
  "completed": "Boolean (Default: false)",
  "createdAt": "ISO String"
}
```

3. Reglas de Negocio e Interfaz

- **Validación:** El formulario no permite el envío si los campos "Título", "Prioridad" o "Categoría" están vacíos.
- **Persistencia:** Los datos se almacenan en el localStorage del navegador, permitiendo que la información se mantenga tras recargar la página.
- **Ordenamiento:** Las tareas nuevas se posicionan al inicio de la lista (unshift).
- **Feedback:** El sistema emite notificaciones visuales temporales tras cada acción exitosa (crear, completar, eliminar).

4. Requerimientos No Funcionales

- **Diseño Responsivo:** Interfaz adaptativa mediante CSS Media Queries para dispositivos móviles.
- **Frontend:** HTML5 semántico, CSS3 con gradientes y animaciones (@keyframes), y JavaScript ES6+.

- **Backend:** Entorno de ejecución Node.js con el framework **Express v5.x** para la exposición del servicio en el puerto **3000**.
- **Dependencias:** Uso de uuid para identificadores únicos y nodemon para entorno de desarrollo.

5. Rutas del Servidor (API Potencial)

Basado en tu server.js, los endpoints base a documentar serían:

- **Base URL:** http://localhost:3000
- **Entry Point:** server.js (instancia el servidor).
- **Lógica de Rutas:** Encapsulada en ./src/app.

4. Metodología de Desarrollo y Control de Versiones (Git Flow)……

Para este proyecto se utiliza Git como sistema de control de versiones distribuido, implementando una adaptación del modelo Git Flow para asegurar un flujo de trabajo organizado y colaborativo.

4.1. Estrategia de Ramas

Se define una estructura jerárquica de ramas para separar el código estable del código en desarrollo:

- **Ramas Principales:**
  - **main:** Contiene el código de producción totalmente estable y funcional. Cada actualización en esta rama representa una versión terminada del sistema.
  - **develop:** Es la rama de integración donde se consolidan todas las nuevas funcionalidades antes de pasar a producción.
- **Ramas de Apoyo:**
  - **feature/ (Funcionalidades):** Ramas temporales creadas a partir de develop para desarrollar tareas específicas (ej. feature/crud-tareas o feature/interfaz-estilos). Al finalizar, se reintegran a develop y se eliminan.
  - **hotfix/ (Correcciones críticas**): Ramas de emergencia que nacen de main para solucionar errores graves en producción sin interrumpir el flujo de desarrollo actual.

4.2. Registro de Commits y Merges

El historial del proyecto se mantiene limpio y descriptivo siguiendo estas convenciones:

- **Estándar de Commits (Conventional Commits):** Cada mensaje debe describir claramente el cambio realizado siguiendo el formato tipo: descripción.
  - **feat:** para nuevas funcionalidades (ej. feat: agregar persistencia en localStorage).
  - **fix**: para corrección de errores (ej. fix: validación de fecha en formulario).

    - **docs:** para cambios en la documentación.
    - **style:** para cambios en el diseño o CSS sin afectar la lógica.
- **Gestión de Merges (Integraciones):**
    - Las integraciones de ramas feature hacia develop se realizan mediante Pull Requests (PR), permitiendo la revisión del código antes de la fusión.
    - Se evita el uso indiscriminado de "Fast-forward" para conservar la trazabilidad de cuándo y quién integró cada funcionalidad en el historial del repositorio.

5. Arquitectura de Diseño Técnico……

Esta sección detalla los componentes tecnológicos y la lógica estructural que permiten el funcionamiento del Sistema de Gestión de Tareas.

5.1. Stack Tecnológico

El proyecto utiliza un stack basado en JavaScript para garantizar una integración fluida entre el cliente y el servidor:

- **Frontend:** HTML5, CSS3 (Custom Properties y Animaciones) y JavaScript Vanilla (ES6+) para la manipulación del DOM y lógica de persistencia.
- **Backend: Node.js** con el framework **Express (v5.2.1)** para la gestión del servidor y el enrutamiento.
- **Entorno de Desarrollo: Nodemon** para el reinicio automático del servidor y **UUID** para la generación de identificadores únicos en el backend.
- **Persistencia:** localStorage del navegador para almacenamiento inmediato de datos en el lado del cliente.

5.2. Estructura de Datos

El sistema maneja un objeto central de tipo Task. A continuación se detalla la interfaz de datos utilizada en la lógica del frontend:

Atributo	**Tipo de Dato**	**Descripción**
id	Number / String	Identificador único generado por Date.now() o uuid.
title	String	Nombre descriptivo de la tarea (Obligatorio).
description	String	Detalles adicionales del requerimiento.

Atributo	Tipo de Dato	Descripción
priority	Enum	Nivel de urgencia: alta, media, o baja.
category	Enum	Clasificación: personal, trabajo, estudios, otros.
completed	Boolean	Estado de la tarea (Default: false).
createdAt	ISO Date	Registro temporal de la creación del objeto.

5.3. Diagrama de Flujo

El flujo operativo del sistema sigue un modelo de eventos reactivos desde la interfaz de usuario:

1. **Inicio:** El sistema carga las tareas existentes desde el localStorage al evento DOMContentLoaded.
2. **Entrada de Datos:** El usuario completa el formulario y se validan los campos obligatorios.
3. **Procesamiento:** * Se crea un objeto JSON con la estructura definida.
    - Se inserta en el arreglo global tasks mediante unshift (orden descendente).
4. **Sincronización:** Se ejecuta saveTasksToStorage() para actualizar el almacenamiento local y se vuelve a renderizar la lista en el DOM.
5. **Interacción:** Si el usuario marca como completada o elimina una tarea, el flujo regresa al paso de sincronización.

[Inicio]

↓

[Carga de tareas desde localStorage (DOMContentLoaded)]

↓

[Entrada de Datos: Usuario completa formulario]

↓

[Validación de campos obligatorios]

↓

┌──────────────────┐

│ ¿Datos válidos? │

└──────────────────┘

│

NO │ SÍ

↓ │ ↓

[Volver a Entrada] [Procesamiento]

↓

[Crear objeto JSON]

↓

[Insertar en arreglo tasks (unshift)]

↓

[Sincronización]

↓

[saveTasksToStorage() + actualizar localStorage]

↓

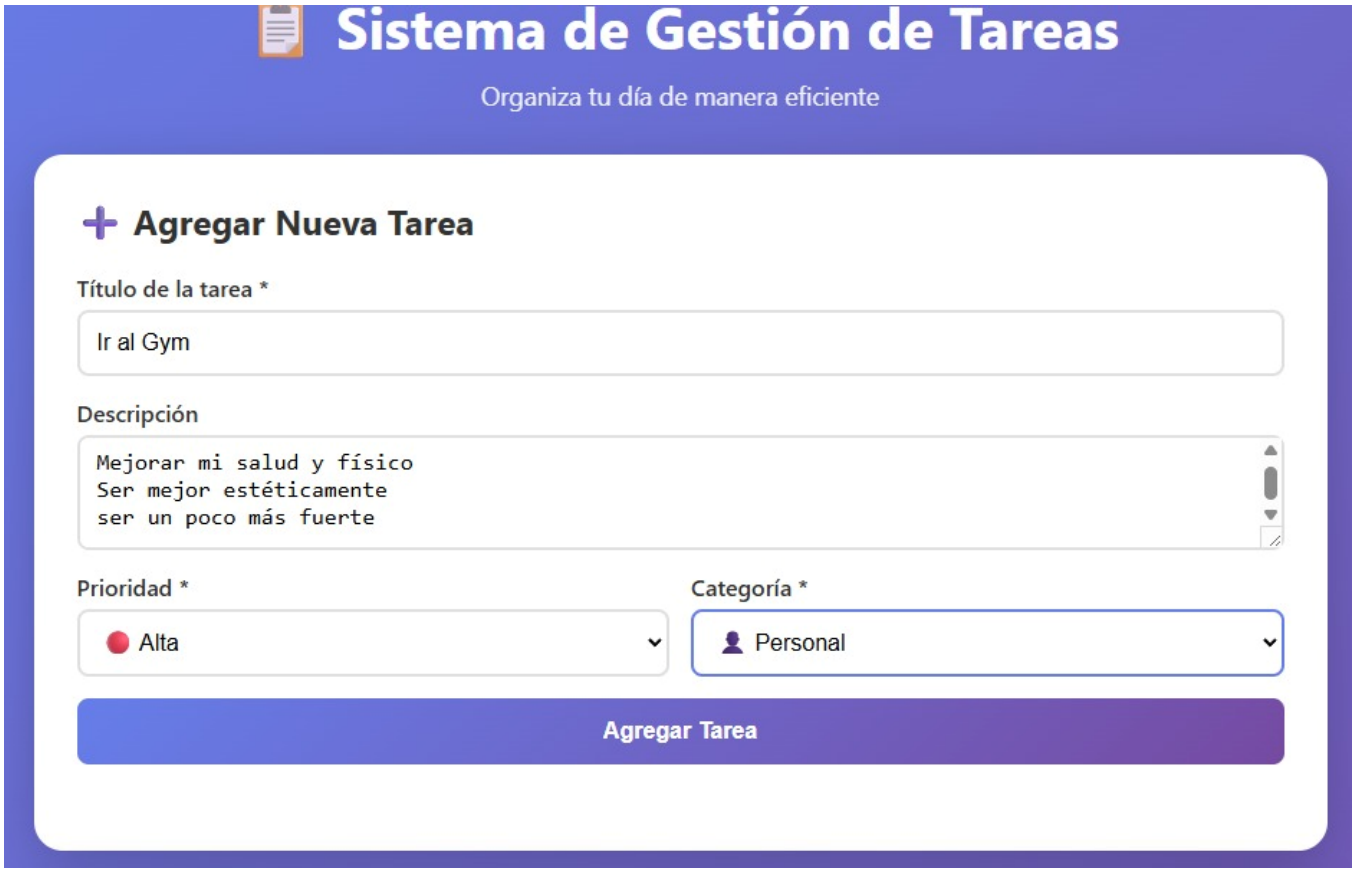
[Renderizar lista en el DOM]

↓

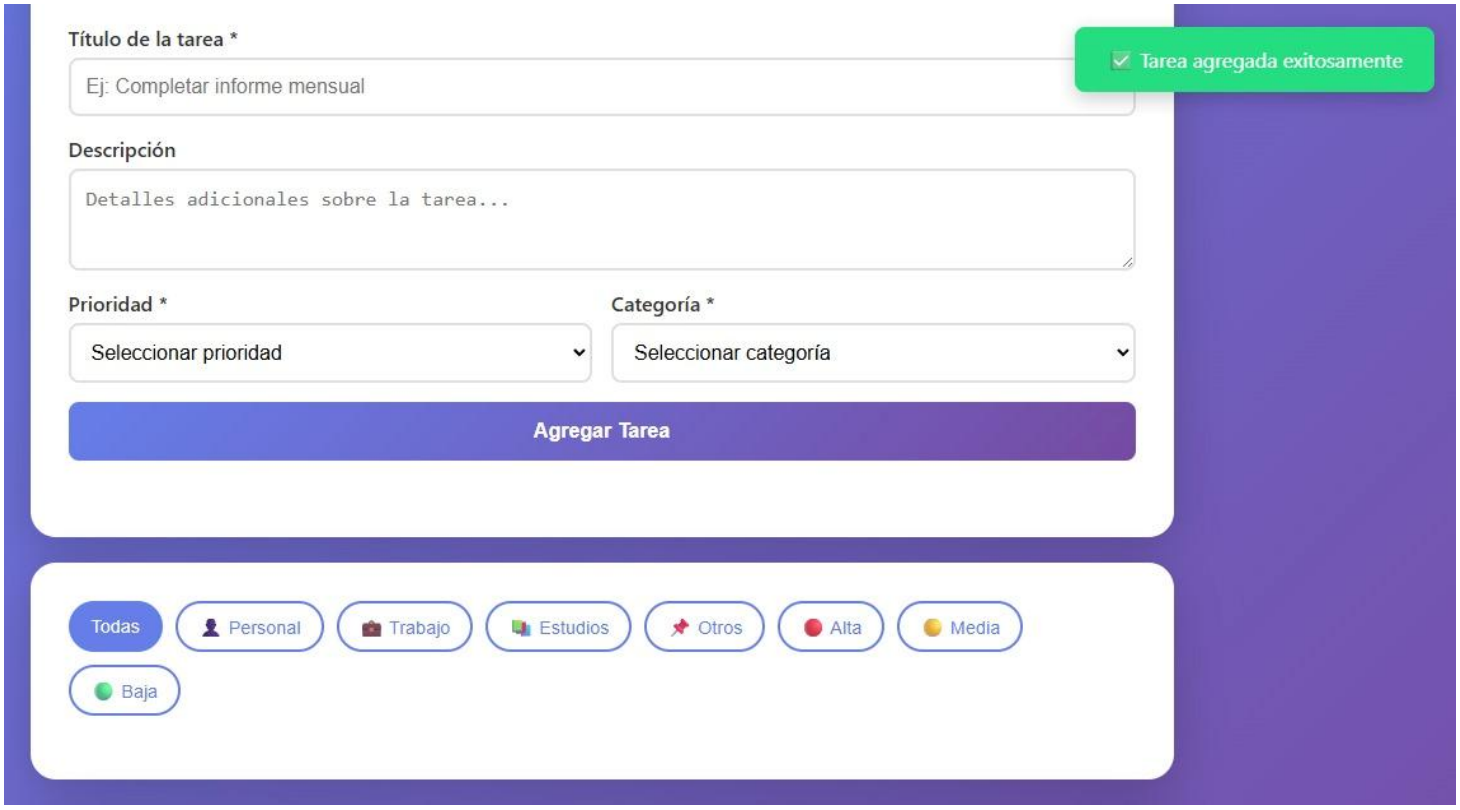
```
┌──────────────────────────────────┐
│ ¿Usuario interactúa?             │
│ (completar / eliminar)           │
└──────────────────────────────────┘
                │
        SÍ      │      NO
        ↓       │      ↓
[Volver a Sincronización]   [Fin]
```

6.Implementación y Pruebas……

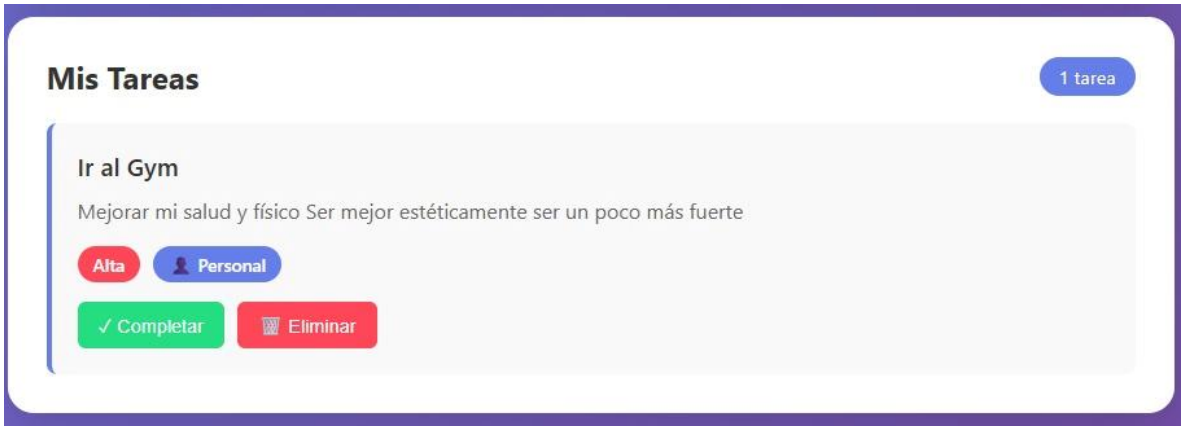
1. INTERFAZ GRAFICA DEL INDEX



- **PRUEBA DE AÑADIR TAREA**



3.TAREA AÑADIDA



7.CONCLUCIONES DEL PROYECTO……

Para cerrar tu documentación del Sistema de Gestión de Tareas, he redactado las conclusiones enfocándome en los objetivos que has alcanzado con el desarrollo técnico que presentaste (Node.js, Express, HTML5 y persistencia local).

Conclusiones

Tras el diseño, desarrollo e implementación del Sistema de Gestión de Tareas, se han extraído las siguientes conclusiones clave sobre el proyecto:

6.1. Eficiencia en la Gestión de Información

Se logró desarrollar una herramienta capaz de organizar tareas de manera jerárquica mediante el uso de prioridades y categorías. La implementación de filtros dinámicos permite al usuario final reducir la carga cognitiva al visualizar únicamente la información relevante en el momento oportuno, cumpliendo con el objetivo de optimizar la productividad personal.

6.2. Robustez y Persistencia de Datos

La arquitectura adoptada garantiza que la información no se pierda tras el cierre de la sesión o la recarga del navegador. Mediante el uso estratégico de Web Storage (localStorage), el sistema ofrece una experiencia de usuario fluida y persistente sin la necesidad de una infraestructura de base de datos compleja para esta primera fase, validando la eficacia del desarrollo en el lado del cliente (Client-side).

6.3. Escalabilidad Técnica

El proyecto queda preparado para una transición hacia un modelo de Full-Stack completo. Al contar ya con un servidor base en Express y una estructura de dependencias definida en package.json, la integración futura de una base de datos centralizada (como MongoDB o PostgreSQL) y la autenticación de usuarios es técnicamente viable con cambios mínimos en la lógica actual.

6.4. Adaptabilidad y Experiencia de Usuario

La integración de un diseño responsivo y notificaciones visuales mediante CSS y JavaScript mejora significativamente la interactividad. El sistema no solo cumple una función técnica, sino que ofrece una interfaz intuitiva que se adapta a diferentes dispositivos, asegurando que la gestión de tareas sea accesible desde cualquier entorno.